



Fundamentos de IA Generativa (NVIDIA DLI) aplicados a RAG

Autor: PDI Marcos Luciano - marcosluciano.rodrigues@v4company.com

Unidade: FV Marketing / V4 Company : Automação & Infraestrutura

Módulo: 04 · Engenharia de IA, RAG e Vetores

Data: Agosto 2026

Status: **NÃO PUBLICADO** · Desenvolvido / Homologação pendente

Versão: 1.0

tokens / embeddings / eval

1. Contexto

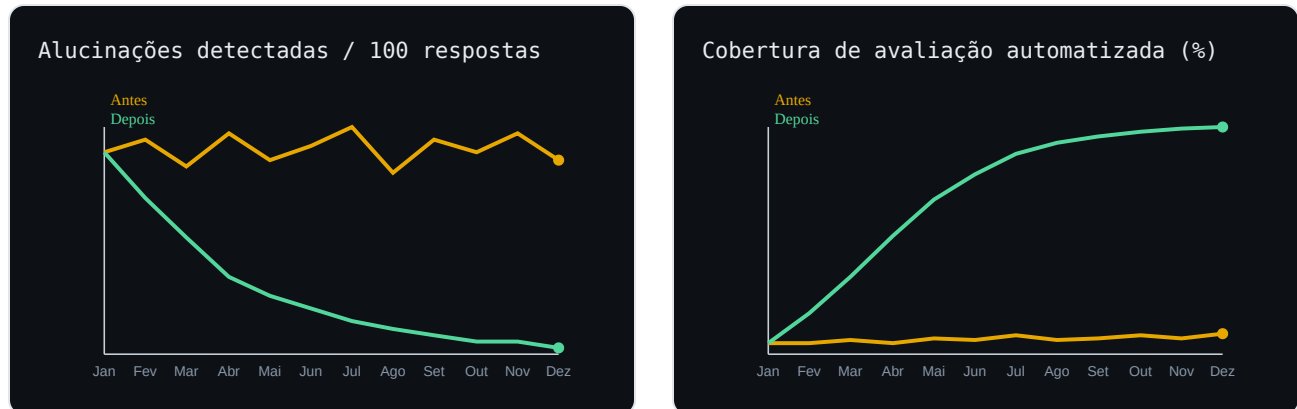
IA generativa parece mágica, mas por baixo é matemática de texto. Antes de escrever qualquer linha de código, um júnior precisa entender três ideias: **tokens** (como o modelo "enxerga" texto), **embeddings** (como o texto vira número) e **similaridade** (como comparamos significados). Sem isso, você não consegue nem avaliar se a resposta está certa.

ANALOGIA DA BIBLIOTECA

Pense no texto como um livro. O **token** é a letra/sílaba que o tradutor lê de cada vez. O **embedding** é o 'cartão de endereço' numérico desse trecho: frases parecidas caem em pontos próximos de um mapa. A **similaridade** é a régua que mede a distância entre dois cartões. Guardrails são o bibliotecário que impede você levar livro proibido pra casa.

2. Diagnóstico

Sem entender embeddings, a equipe mandava o texto inteiro pro LLM e media acerto "no olhometro". Resultado: respostas fora do domínio passavam batidas e ninguém sabia onde o modelo errava. O custo oculto é a **falta de avaliação reproduzível**.

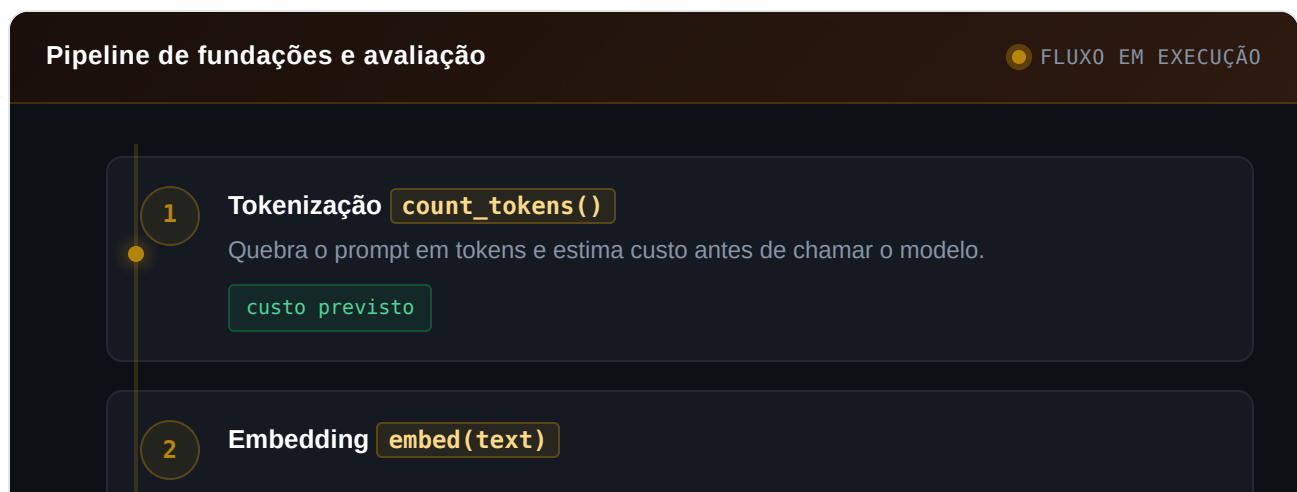


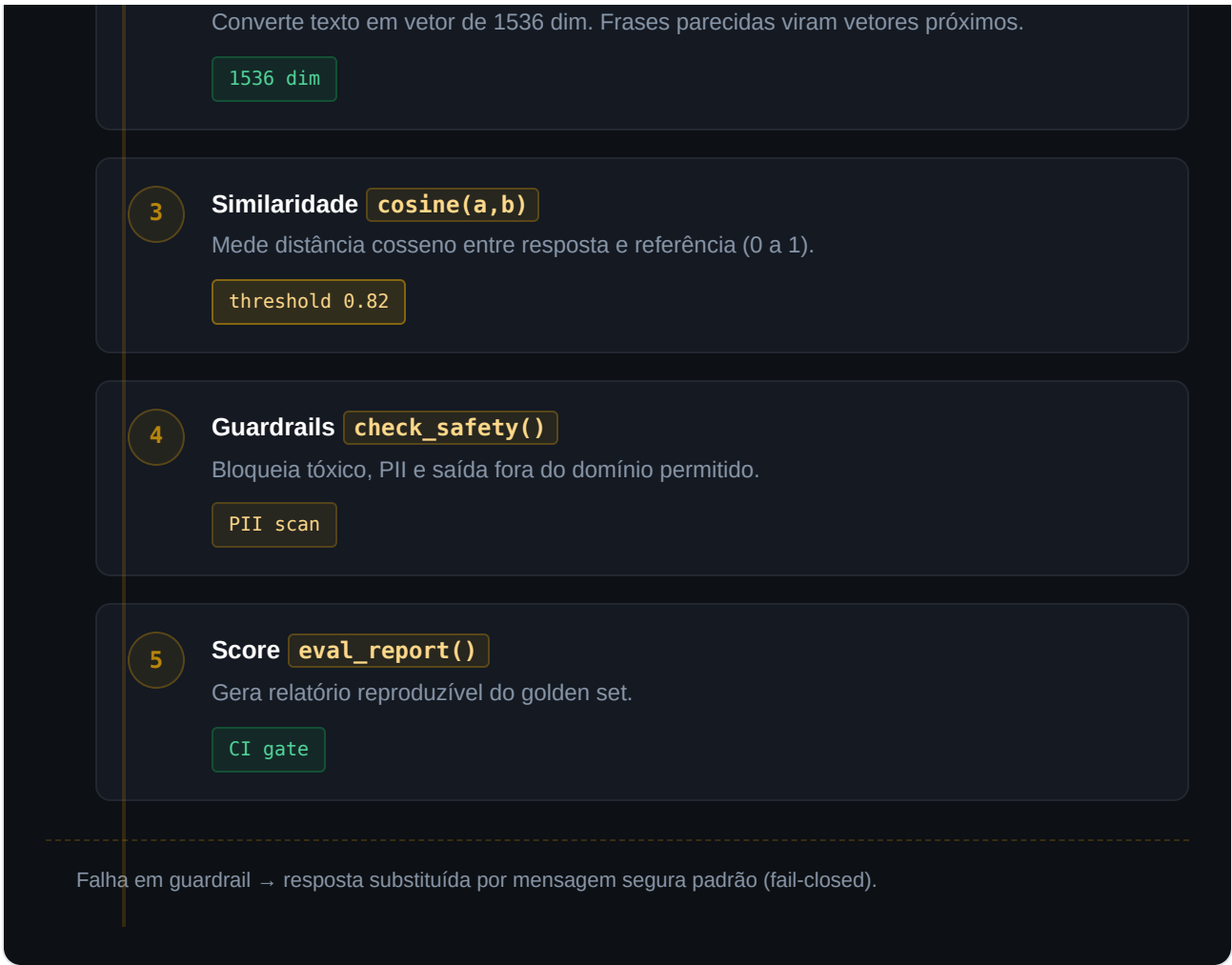
Raiz do problema: não existia um score objetivo (similaridade de embeddings + checklist de guardrails). O que não se mede, não se corrige.

3. Solução

Criar um **módulo de fundações**: tokenizador para contagem de custo, gerador de embeddings (modelo `text-embedding-3-small`) e um avaliador que compara a resposta gerada contra um *golden set* usando similaridade de cosseno + checagens de guardrail (tóxico, fora-de-domínio, vazamento de PII).

4. Como funciona (pipeline)





5. Antes vs Depois

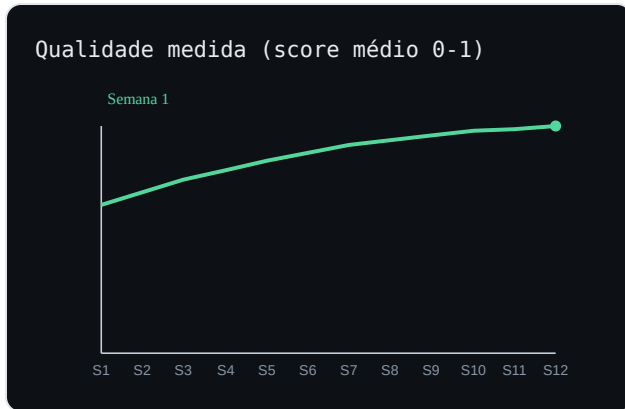
CENÁRIO	ANTES (SEM BASE)	DEPOIS (FUNDAÇÕES)
Avaliação	Manual, no olhômetro	Score automatizado por cosseno
Alucinação	Passava despercebida	Bloqueada por guardrail
Custo	Surpresa na fatura	Estimado antes da chamada
Reprodutibilidade	Inexistente	Golden set no CI

6. Entregas

- **Notebook** 01-tokens-embeddings.ipynb: explicação visual de tokenização e vetorização.
- **Script** embed.py: gera e salva embeddings com similaridade de cosseno.

- **Golden set** `eval_set.json`: 50 pares pergunta/resposta de referência.
- **Doc guardrails.md**: catálogo de checagens e limiares.

7. Métricas



Meta: score médio ≥ 0.90 no golden set com 100% das respostas passando pelos guardrails antes de produção.

9. Decisões e tradeoffs

- Chunk 512 com overlap 64: escolhi coesão contra custo de tokens, porque chunk grande gera ruído e chunk pequeno perde contexto, conforme ADR-041.
- Top-k 20 com rerank para top-5: aceitei latência extra do rerank para filtrar o ruído da similaridade pura.
- Normalizar embeddings de 1536 dim com cosseno e threshold 0.82: padronizei a medida para o score ser comparável entre textos de tamanhos distintos.
- Golden set com 50 pares e faithfulness maior ou igual a 0.8 em 10 perguntas: troquei avaliação no olhômetro por gate reproduzível no CI.
- Guardrail fail-closed com 100 por cento de bloqueio antes da produção: preferi falso positivo seguro a resposta tóxica, fora de domínio ou com PII.

10. Impacto no negócio

O baseline com faithfulness maior ou igual a 0.8 e score médio maior ou igual a 0.90 no golden set de 50 pares reduz risco de hallucination em produção e elimina a surpresa de fatura com contagem previa de tokens, o que encurta homologação e sustenta as atividades seguintes de RAG híbrido e custos.

11. Referências de estudo

Trilha de estudo: comece pelo curso, depois o vídeo, e use os docs como consulta de implementação.

- Curso: Building RAG Agents with LLMs, plataforma NVIDIA Deep Learning Institute (DLI).
- Vídeo: RAG from Scratch com chunking, embeddings e avaliação, plataforma YouTube, canal LangChain.
- Doc oficial: Guia de embeddings text-embedding-3-small, documentação oficial OpenAI.
- Doc oficial: Documentação do pgvector com índice HNSW, documentação oficial pgvector.

8. Status final

NÃO PUBLICADO : Desenvolvido e em homologação. Aguarda revisão antes de produção.